

```
Start here X Safety Algorithm.c X
1  #include <stdio.h>
2  int main() {
3      int n, m, i, j, k;
4
5      printf("Enter number of processes: ");
6      scanf("%d", &n);
7
8      printf("Enter number of resources: ");
9      scanf("%d", &m);
10
11     int alloc[n][m], max[n][m], need[n][m];
12     int avail[m], finish[n], safe[n];
13
14     printf("Enter Allocation Matrix:\n");
15     for(i = 0; i < n; i++) {
16         for(j = 0; j < m; j++) {
17             scanf("%d", &alloc[i][j]);
18         }
19     }
20
21     printf("Enter Maximum Demand Matrix:\n");
22     for(i = 0; i < n; i++) {
23         for(j = 0; j < m; j++) {
24             scanf("%d", &max[i][j]);
25         }
26     }
27
28     printf("Enter Available Resources:\n");
29     for(j = 0; j < m; j++) {
30         scanf("%d", &avail[j]);
31     }
32
33     for(i = 0; i < n; i++) {
34         for(j = 0; j < m; j++) {
35             need[i][j] = max[i][j] - alloc[i][j];
36         }
37     }
38
39     for(i = 0; i < n; i++) {
40         finish[i] = 0;
41     }
42
43     int count = 0;
44
45     while(count < n) {
46         int found = 0;
47
48         for(i = 0; i < n; i++) {
49
50             if(finish[i] == 0) {
51
52                 int possible = 1;
53
```

```

53
54
55         for(j = 0; j < m; j++) {
56             if(need[i][j] > avail[j]) {
57                 possible = 0;
58                 break;
59             }
60
61         if(possible) {
62
63             for(k = 0; k < m; k++) {
64                 avail[k] += alloc[i][k];
65             }
66
67             safe[count] = i;
68             finish[i] = 1;
69             count++;
70             found = 1;
71         }
72     }
73 }
74
75 if(found == 0)
76     break;
77 }
78
79 if(count == n) {
80     printf("\nSystem is in a safe state.\n");
81     printf("Safe sequence is: ");
82
83     for(i = 0; i < n; i++) {
84         printf("P%d", safe[i]);
85
86         if(i != n - 1)
87             printf(" -> ");
88     }
89 }
90 else {
91     printf("\nSystem is NOT in a safe state.\n");
92 }
93
94 return 0;
95 }
96

```

```
"D:\1WA24CS231\Safetly Algo" X + v
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0)   execution time : 271.055 s
Press any key to continue.
```

Output
1WA24CS231

Deadlock Detection

```
#include <stdio.h>
int main() {
    int n, m;
    int i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int alloc[n][m];
    int req[n][m];
    int avail[m];
    int finish[n];
    int safe[n];
    printf("Enter allocation matrix:\n");
    for(i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("Enter request matrix:\n");
    for(i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++) {
            scanf("%d", &req[i][j]);
        }
    }
    printf("Enter available resources: ");
    for(i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }
    for(i = 0; i < n; i++) {
        finish[i] = 0;
    }
    int count = 0;
    while(count < n) {
        int found = 0;
        for(i = 0; i < n; i++) {
            if(finish[i] == 0) {
                int possible = 1;
                for(j = 0; j < m; j++) {
                    if(req[i][j] > avail[j]) {
                        possible = 0;
                    }
                }
                if(possible == 1) {
                    for(j = 0; j < m; j++) {
                        avail[j] = avail[j] + alloc[i][j];
                    }
                    safe[count] = i;
                    count++;
                    finish[i] = 1;
                }
            }
        }
    }
}
```

```

        finish[i] = 1;
        found = 1;
    }
}
if(found == 0) {
    break;
}
}
if(count == n) {
    printf("System is in safe state\n");
    printf("Safe Sequence is: ");
    for(i = 0; i < n; i++) {
        printf("P%d ", safe[i]);
    }
}
else {
    printf("System is NOT in safe state");
}
return 0;
}

```

Output

```

Enter number of processes: 5
Enter number of resources: 3
Enter allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter available resources: 0 0 0
System is in safe state
Safe Sequence is: P0 P2 P3 P4 P1
Process returned 0 (0x0)    execution time : 67.779 s
Press any key to continue.

```